Documentation Algorithmique

Projet Web Scraping + Classification NLP

Master 1 IdL — Algorithmique Python S2 — 2025/2026

1. PARTIE 1 — WEB SCRAPING

1.1 Structures de donnees manipulees

Structure	Type Python	Contenu	Complexite acces
Page HTML parsee	lxml.html.HtmlElement	Arbre DOM complet de la page	O(1)
Metadonnees article	dict	{titre, date, chapeau, label}	O(1)
Texte extrait	str	Corps de l'article nettoye sans menus	O(n)
Un article	dict	{url, titre, date, chapeau, texte, label}	O(1)
URLs visitees	set	Ensemble des URLs deja scrapees (deduplication)	O(1) moy.
Dataset complet	list[dict]	50 articles, 5 categories x 10	O(n)
Fichier de sortie	CSV UTF-8	6 colonnes : url|titre|date|chapeau|texte|label	O(n)

1.2 Fonctions et complexite

Fonction	Entree	Sortie	Action sur les donnees	Complexite
extraire_label_depuis_url(url)	str	str|None	Decoupe l'URL, cherche le label dans les sections	O(k)
extraire_metadonnees(tree, url)	lxml, str	dict	Parcourt le DOM via XPath, construit un dict	O(n)
nettoyer_texte(texte)	str	str	Supprime \n, \t, espaces multiples par regex	O(n)
est_parasite(ligne)	str	bool	Cherche mots-cles parasites dans la ligne	O(k)
extraire_texte_article(tree)	lxml	str	XPath //article//p, filtre parasites, joint les lignes	O(n)
scraper_section(url, label, nb)	str, str, int	list[dict]	Requetes HTTP, parse HTML, appelle fonctions ci-dessus	O(nb*T)
sauvegarder_csv(donnees, fich)	list, str	fichier CSV	Ecrit chaque dict en ligne CSV UTF-8	O(n)
afficher_statistiques(fich)	str	terminal	Lit CSV, compte par label, calcule moyenne	O(n)

1.3 Correction extraction texte

Probleme identifie : XPath //p//text() extrayait tout le contenu de la page (menus, newsletters, listes de videos) identique sur chaque article. Correction appliquee : //article//p//text() cible uniquement le corps editorial. Double protection par liste MOTS_PARASITES.

```
# AVANT (probleme de duplication)
paragraphes = tree.xpath('//p//text()')

# APRES (correction)
paragraphes = tree.xpath('//article//p//text()')
```

2. PARTIE 2 — CLASSIFICATION CAMEMBERT

2.1 Dataset AVANT tokenisation

Resultat observe dans Colab : 50 articles, 5 categories x 10. GPU disponible : False (CPU utilise).

Propriete	Valeur observee
Type	pandas.DataFrame
Shape	(50, 6)
Colonnes	url, titre, date, chapeau, texte, label
Colonne texte_complet	str — concatenation titre + texte
Colonne label_id	int64 — encode par LabelEncoder
Mapping labels	economie=0, politique=1, sante=2, sciences=3, sport=4
Split train/test	40 exemples train / 10 exemples test (80/20, stratifie)

2.2 Dataset APRES tokenisation

Resultat observe dans Colab : input_ids shape torch.Size([1, 128]), attention_mask shape torch.Size([1, 128]).

Cle	Type	Shape observee	Description
input_ids	torch.Tensor	torch.Size([1, 128])	Indices des tokens dans le vocabulaire CamemBERT. Texte tronque a 128 tokens max.
attention_mask	torch.Tensor	torch.Size([1, 128])	1 = token reel, 0 = padding. Indique au modele quels tokens ignorer.
labels	torch.Tensor	torch.Size([N])	Labels encodes int64 pour chaque exemple du batch.

2.3 Sorties du modele — Logits

Resultat observe dans Colab : Structure logits : (10, 5) — 10 exemples de test, 5 classes.

Variable	Type	Shape observee	Description
logits	np.ndarray	(10, 5)	Score brut pour chaque classe, avant softmax. Un vecteur de 5 valeurs par exemple.
y_pred	np.ndarray	(10,)	argmax(logits, axis=1) — indice de la classe la plus probable
y_true	np.ndarray	(10,)	Labels reels du jeu de test
predictions finales	list[str]	(10,)	LabelEncoder.inverse_transform(y_pred) — labels en texte

2.4 Resultats d'entrainement observes

Metrique	Epoch 1	Epoch 2	Epoch 3
Training Loss	1.613327	1.604673	1.593094
Validation Loss	1.604278	1.599443	1.597372
Accuracy	0.40	0.50	0.50
F1-score	0.280000	0.422222	0.366667

Accuracy finale : 50.0% — F1-score final : 36.67%. Ces resultats s'expliquent par la taille reduite du corpus (50 articles, 10 par classe).

2.5 Rapport de classification detaille

Classe	Precision	Recall	F1-score	Support
economie	0.00	0.00	0.00	2
politique	0.50	1.00	0.67	2
sante	0.00	0.00	0.00	2
sciences	0.50	0.50	0.50	2
sport	0.50	1.00	0.67	2
accuracy			0.50	10
macro avg	0.30	0.50	0.37	10
weighted avg	0.30	0.50	0.37	10

2.6 Pipeline complet — fonction predire(texte)

```
ENTREE : texte (str)

1. tokens <- CamembertTokenizer(texte, max_length=128, truncation=True)
   input_ids      : Tensor(1, 128) — indices tokens
   attention_mask : Tensor(1, 128) — 1=token reel, 0=padding

2. sortie <- CamemBERT(input_ids, attention_mask)
   logits : Tensor(1, 5) — score brut pour chaque classe

3. indice <- argmax(logits)

4. label  <- LabelEncoder.inverse_transform([indice])

SORTIE : label (str) — categorie predite
```

2.7 Limites et ameliorations possibles

Le modele converge difficilement avec 50 articles (10 par classe). Pour ameliorer : augmenter le corpus a 200+ articles par classe, augmenter max_length a 256 tokens, augmenter le nombre d'epochs a 5-10.